



Università degli Studi di Napoli
FEDERICO II

A.A. 2025-2026

INGENIERIA DEL SOFTWARE

0 NOTE SUL CORSO	3
1 LEZIONE 1 E 2 --- PROCESSI SOFTWARE E QUALITÀ DEL SOFTWARE	4
Le attività del ciclo di vita del software	4
Il software non è mai davvero finito	4
Stesse attività, organizzazioni diverse	4
Le attività in dettaglio	4
Ingegneria dei requisiti	4
Progettazione del software e architettura	5
Implementazione	5
Verifica e validazione (V&V)	5
Rilascio, esercizio ed evoluzione	5
Testing	5
La qualità del software	5
Che cosa si intende per qualità	6
I modelli di qualità	6
Le tre prospettive sulla qualità	6
ISO/IEC 25010:2023 — Qualità del prodotto	7
Adeguatezza funzionale	7
Affidabilità	7
Efficienza prestazionale	7
Capacità di interazione	7
Sicurezza informatica	7
Compatibilità	8
Manutenibilità	8
Flessibilità	8
Sicurezza (Safety)	8
La qualità in uso	8
Usabilità	9
Sicurezza	9
Flessibilità	9
Misurare la qualità del software	9
Valutare un attributo di qualità	9
Evidenze e limiti della valutazione	10
2 LEZIONE 3 E 4 --- REQUIREMENTS ENGINEERING: ELICITATION AND ANALYSIS	11
Software Requirements	11
Livelli di requisiti	11
Tipi di requisiti	11
Proprietà di un buon requisito	12
Il processo di Requirements Engineering	12
Requirements Elicitation and Analysis	13
Sfide dell'elicitazione	13
Processo di elicitation e analisi	13
Tecniche di elicitation	13
Interviste con gli stakeholder	13
Etnografia	14
Personas	14
Stories	14
Low-fidelity mockups	14
Collocazione nel ciclo di vita	15

3 LEZIONE 5 E 6 --- REQUIREMENTS ENGINEERING: SPECIFICATION	16
Il ciclo di vita del software e il posto della RE	16
Il processo di Requirement Engineering	16
Requirements Specification	17
Requirements e design	17
Approcci alla specifica	17
Specifica in linguaggio naturale (NL)	17
Specifiche strutturate	18
Use Case Diagrams	18
Use Cases	18
Attori	19
Use Cases	19
System Boundary	19
Relazioni nel UCD	19
Errori comuni (Rookie Mistakes)	20
Specifica dettagliata dei Use Cases	21
Contenuto di una descrizione testuale	21
Formati dei use case	21
Template di Cockburn (semplificato)	21
Requirements Validation	22
Requirement checking	22
Tecniche di validazione	22
 4 LEZIONE --- UML: CLASS DIAGRAM E SEQUENCE DIAGRAM	 23
Il Class Diagram e i suoi limiti	23
I Sequence Diagram	23
Confronto con il Class Diagram	23
Sintassi di base	23
Elementi avanzati	24
Autochiamata (self-call)	24
Operatori e frame di interazione	24
Chiamate sincrone e asincrone	24
Corrispondenza tra Sequence Diagram e codice	24
Associazione vs dipendenza	25
Strumenti CASE	25
Sintassi completa del Sequence Diagram	25

Note sul corso

I seguenti appunti sono stati presi durante le lezioni del corso di Tecnologie Web.

Nota:

Lezione 1 e 2 — Processi Software e Qualità del Software

L'ingegneria del software si occupa della **qualità del prodotto software lungo tutto il suo ciclo di vita**. Lo studio dei processi software non parte dai modelli di processo: prima si analizzano le attività che essi organizzano, poiché è solo comprendendo le singole attività che si può capire come esse vengano coordinate in modi diversi.

Le attività del ciclo di vita del software

Un **processo software** è un insieme di attività correlate che porta alla produzione di un sistema software. Non esiste un processo universale che funziona sempre: esistono molti processi diversi, tutti ugualmente validi a seconda del contesto. Tutti questi processi, però, includono in qualche forma le stesse **attività fondamentali**, ed è per questo che possiamo studiarle in modo generale prima di affrontare i modelli di processo.

Le quattro attività fondamentali sono:

- La **specificazione del software** comprende i bisogni degli stakeholder e definisce il comportamento richiesto e i vincoli.
- Lo **sviluppo del software** progetta e implementa una soluzione che risponde a quei bisogni.
- La **validazione del software** valuta se la soluzione supporta l'uso previsto.
- L'**evoluzione del software** adatta il software al mutare di bisogni, dipendenze e condizioni operative.

Queste attività sono complesse e comprendono molte sotto-attività. Come ogni processo creativo, un processo software si basa su **persone** che prendono **decisioni** e **giudizi**, e include tipicamente anche attività aggiuntive come la **pianificazione** e il **monitoraggio del progetto**.

Il software non è mai davvero finito

Lo sviluppo iniziale costituisce solo una parte del ciclo di vita del software. Requisiti, piattaforme, dipendenze e minacce cambiano continuamente, quindi il software deve essere costantemente mantenuto, adattato e fatto evolvere. Ne deriva che **progettare per il cambiamento è un problema centrale dell'ingegneria del software**, non un aspetto marginale.

Stesse attività, organizzazioni diverse

Un processo software definisce **come attività, ruoli e artefatti vengono organizzati e coordinati**. Le attività fondamentali sono sempre le stesse; ciò che cambia da processo a processo è il modo di organizzarle. Per questo motivo le si studia prima singolarmente e i diversi modi di organizzarle in processi completi si vedono alla fine del corso.

Le attività in dettaglio

Ingegneria dei requisiti

L'obiettivo è comprendere i bisogni degli stakeholder e definire ciò che il software deve realizzare, inclusi i requisiti di qualità e i vincoli. Il lavoro consiste nell'elicitarli e analizzare i bisogni insieme agli stakeholder, per poi registrarli con forme adeguate come specifiche, casi d'uso o user story; a questo punto si definiscono i criteri di accettazione e si risolvono le ambiguità. Poiché i requisiti cambiano nel tempo, è necessario mantenere i legami tra requisiti, decisioni di progetto e test man mano che evolvono.

I requisiti vengono raccolti tramite interviste con gli stakeholder, personas, storie e scenari; vengono specificati usando casi d'uso, linguaggio naturale, modelli di dominio e mock-up.

Progettazione del software e architettura

L'obiettivo è progettare una **struttura adeguata (architettura)** del software, che supporti il comportamento richiesto e gli attributi di qualità. L'architettura individua i componenti principali, le loro responsabilità e le loro interazioni, mentre la progettazione dettagliata definisce interfacce, classi e le relative collaborazioni. La progettazione della UI e i prototipi servono a valutare l'interazione con gli utenti.

Le decisioni di progetto comportano sempre **compromessi (trade-off)**: ad esempio, puntare sulle prestazioni può sacrificare la modificabilità. Poiché ogni decisione ha una motivazione, modelli e record delle decisioni spiegano la soluzione e la sua ratio; UML supporta viste selezionate del progetto.

La progettazione si articola su due livelli:

- Nella **progettazione di sistema** i requisiti vengono allocati a sotto-sistemi software e i sotto-sistemi a risorse hardware, usando pattern architetturali.
- Nella **progettazione software e della UI** si definiscono gli oggetti necessari a realizzare ogni sotto-sistema, usando design pattern, usability engineering e wireframing ad alta fedeltà.

Implementazione

L'obiettivo è **costruire software funzionante**, raffinando e verificando le decisioni di progetto. Si implementa il comportamento richiesto gestendo gli errori esplicitamente, e si scrive codice che altri sviluppatori possano comprendere, testare e modificare. Tramite code review e analisi statica si esamina l'implementazione, mentre build e test di regressione vengono automatizzati.

Le linee guida del Clean Code sono **euristiche** da valutare nel contesto, non regole assolute. Un punto critico attuale: gli sviluppatori restano responsabili della revisione e della validazione dei contributi assistiti dall'IA, anche quando il codice viene generato automaticamente.

Verifica e validazione (V&V)

La domanda alla base della V&V è: gli artefatti sviluppati soddisfano davvero i bisogni degli utenti? Si distinguono due attività complementari. La **verifica** chiede se il sistema è conforme alla sua specifica — cioè “abbiamo costruito la cosa giusta *nel modo giusto*?” — e si fa tipicamente con il testing del programma, eseguendo il software in un ambiente controllato e verificando che il comportamento sia corretto rispetto alle specifiche. La **validazione** chiede invece se il sistema soddisfa le aspettative dei clienti — “abbiamo costruito la cosa giusta?” — e si fa tipicamente definendo ed eseguendo test di accettazione.

La V&V si applica anche a requisiti e progettazione, non solo al software eseguibile, e continua durante l'evoluzione del sistema. I test di accettazione contribuiscono alla validazione, ma con una precisazione importante: superare una suite di test fornisce **evidenze solo nell'ambito di quei test**, non una garanzia generale.

Rilascio, esercizio ed evoluzione

Il **rilascio** prepara e mette il software nel suo ambiente di destinazione, mentre l'**esercizio** mantiene il servizio disponibile e fornisce evidenze sul comportamento in uso. La **manutenzione** corregge i difetti e adatta il software a bisogni, dipendenze e ambienti in cambiamento: serve sia a riparare errori non scoperti nelle fasi precedenti, sia ad adattare il software a cambiamenti nei requisiti o nell'ambiente. Poiché ogni cambiamento comporta rischi, controlli automatici di rilascio e procedure di ripristino aiutano a gestirli.

Nel corso si introdurranno CI/CD, osservabilità e le pratiche a supporto dell'evoluzione; questi temi sono approfonditi nel corso magistrale di Software Project Management and Evolution.

Testing

Il testing serve a garantire che il software soddisfi i clienti e comprende: ispezioni del codice, test funzionali (di unità, di integrazione, di sistema) e test di usabilità.

La qualità del software

Che cosa si intende per qualità

La qualità di un prodotto riguarda diverse proprietà la cui importanza dipende dall'uso previsto, e questo vale per qualsiasi prodotto, non solo per il software:

- Un'**automobile** è di qualità se si guasta raramente, consuma poco e ha bassi costi di manutenzione.
- Un **orologio** è di qualità se è molto preciso, resistente ad acqua e polvere e resistente ai graffi.
- Delle **parti meccaniche** sono di qualità se hanno basse tolleranze, aderiscono alle specifiche e resistono all'usura.

Per il software la questione è più sfumata, come mostrano tre esempi classici.

L'inverse square root di Quake III Arena

Il codice seguente è l'implementazione dell'**inverse square root** attribuita a John Carmack:

```
float Q_rsqrt(float number){
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;
    i = 0x5f3759df - ( i >> 1 );
    y = * ( float * ) &i;
    y = y * ( threehalfs - ( x2 * y * y ) );

    return y;
}
```

L'inverse square root ($1/\sqrt{x}$) è molto usato in computer grafica, ad esempio per calcolare gli angoli di incidenza e riflessione. L'approssimazione di Carmack era circa 4 volte più veloce di `(float)(1.0/sqrt(x))`, e molti la considerano un esempio di grande programmazione. Ma il codice è quasi illeggibile: riuscire a capire come funziona è tutt'altro che immediato. Ne deriva la domanda: **essere efficienti è alta qualità?**

OpenBSD e Vim

Alcuni considerano **OpenBSD** software di alta qualità perché è un sistema operativo molto stabile e comparativamente difficile da violare — ma allora **essere molto sicuri contro gli attaccanti è alta qualità?** Analogamente, **Vim** è considerato da molti il miglior editor di testo: gli esperti lavorano estremamente veloce con esso, quindi sarebbe software di alta qualità; altri però dicono che è troppo difficile da imparare, e quindi di bassa qualità. La conclusione è che **non esiste «una sola qualità»**, ma molti approcci e punti di vista diversi.

I modelli di qualità

Un **modello di qualità** offre agli stakeholder un **vocabolario condiviso** per discutere requisiti e valutazione. Le priorità e i criteri di valutazione rilevanti dipendono dal prodotto e dai suoi stakeholder. Gli standard di riferimento sono tre:

- **ISO/IEC 25010:2023** descrive la qualità del prodotto attraverso nove caratteristiche.
- **ISO/IEC 25019:2023** tratta la qualità in uso (*quality in use*).
- **ISO/IEC 25002:2024** illustra il framework e l'uso dei modelli di qualità.

Le tre prospettive sulla qualità

La qualità del software si può osservare da tre prospettive distinte. La **qualità interna** riguarda le proprietà esaminate senza eseguire il software, come le dipendenze e la struttura del codice. La **qualità esterna** riguarda le proprietà osservate durante l'esecuzione, come il tempo di risposta e il comportamento in caso di guasto. La **qualità in uso** riguarda i risultati per le persone e le organizzazioni che usano il sistema in un contesto specificato.

Le pratiche di processo aiutano a raggiungere e valutare la qualità, ma la loro adozione da sola non basta: deve essere accompagnata da **evidenze sul prodotto e sul suo uso**.

ISO/IEC 25010:2023 — Qualità del prodotto

Lo standard descrive la qualità del prodotto attraverso **nove caratteristiche**, ciascuna ulteriormente suddivisa in sotto-caratteristiche.

Adeguatezza funzionale

La **Functional Suitability** è il grado in cui un componente o sistema fornisce funzioni che soddisfano bisogni espliciti e impliciti in condizioni specificate. Si compone di tre sotto-caratteristiche:

- La **completezza funzionale** misura quanto le funzioni fornite coprono tutti i compiti e gli obiettivi utente specificati.
- La **correttezza funzionale** misura quanto il prodotto fornisce risultati accurati quando usato dagli utenti previsti.
- L'**appropriatezza funzionale** misura quanto le funzioni facilitano il raggiungimento dei compiti e degli obiettivi specificati.

Un esempio: un sistema di prenotazione aule deve supportare la cancellazione, applicare correttamente le regole di prenotazione e aiutare gli utenti a trovare un'aula adatta.

Affidabilità

La **Reliability** è il grado in cui il sistema svolge le sue funzioni in condizioni specificate per un periodo di tempo specificato. Le sue sotto-caratteristiche sono:

- L'**assenza di guasti** (*Faultlessness*) misura quanto il sistema svolge le funzioni specificate senza guasti nel funzionamento normale.
- La **disponibilità** (*Availability*) misura quanto il sistema è operativo e accessibile quando richiesto.
- La **tolleranza ai guasti** (*Fault tolerance*) misura quanto il sistema opera come previsto nonostante la presenza di guasti hardware o software.
- La **ripristinabilità** (*Recoverability*) riguarda il ripristino dei dati affetti e il ritorno allo stato operativo richiesto dopo un'interruzione o un guasto.

Efficienza prestazionale

La **Performance Efficiency** riguarda tempi di risposta, throughput e uso delle risorse in condizioni operative specificate. Si articola in:

- Il **comportamento temporale** (*Time behaviour*) misura quanto tempi di risposta e throughput soddisfano i requisiti.
- L'**utilizzo delle risorse** (*Resource utilization*) misura quanto quantità e tipi di risorse usate soddisfano i requisiti.
- La **capacità** (*Capacity*) misura quanto i limiti massimi dei parametri del sistema soddisfano i requisiti.

Capacità di interazione

L'**Interaction Capability** riguarda le caratteristiche del prodotto che supportano gli utenti nell'interagire con il sistema per completare i compiti. Include, tra le altre, le seguenti sotto-caratteristiche:

- La **riconoscibilità** (*Recognizability*) misura quanto gli utenti possono riconoscere se il sistema è adatto ai loro bisogni.
- L'**apprendibilità** (*Learnability*) misura quanto le funzioni possono essere apprese da utenti specificati entro un tempo specificato.
- L'**operabilità** (*Operability*) misura quanto il prodotto è facile da usare e controllare.
- La **protezione dagli errori utente** (*User error protection*) misura quanto il sistema previene gli errori.
- L'**inclusività** (*Inclusivity*) misura quanto il sistema può essere usato da persone di contesti diversi: età, abilità, culture, etnie, lingue, generi, situazioni economiche, ecc.

Sicurezza informatica

La **Security** è il grado in cui un sistema si difende da attacchi di attori malevoli e protegge informazioni e dati applicando adeguati meccanismi di autorizzazione. Le sue sotto-caratteristiche principali sono:

- La **riservatezza** (*Confidentiality*) misura quanto i dati sono accessibili solo a chi è autorizzato.

- L'**integrità** (*Integrity*) misura quanto stato e dati sono protetti da modifiche o cancellazioni non autorizzate.
- La **non ripudiabilità** (*Non-repudiation*) misura quanto azioni o eventi possono essere provati, così da non poter essere negati in seguito.
- L'**accountability** misura quanto le azioni di un'entità possono essere ricondotte univocamente a essa.
- L'**autenticità** (*Authenticity*) misura quanto l'identità di un soggetto o risorsa può essere provata come quella dichiarata.

Compatibilità

La **Compatibility** è il grado in cui un sistema può scambiare informazioni con altri prodotti, sistemi o componenti e/o svolgere le sue funzioni condividendo lo stesso ambiente e le stesse risorse comuni con altri sistemi. Si compone di:

- La **co-esistenza** (*Co-existence*) misura quanto un prodotto svolge le sue funzioni in modo efficiente condividendo ambiente e risorse con altri prodotti, senza impatti dannosi su questi ultimi.
- L'**interoperabilità** (*Interoperability*) misura quanto un sistema può scambiare informazioni con altri prodotti e usare reciprocamente le informazioni scambiate.

Manutenibilità

La **Maintainability** è il grado di efficacia ed efficienza con cui un prodotto può essere modificato per migliorarlo, correggerlo o adattarlo a cambiamenti di ambiente e requisiti. Le sue sotto-caratteristiche riguardano altrettanti obiettivi di progetto:

- La **modularità** (*Modularity*) limita l'impatto dei cambiamenti tra i componenti.
- Il **riuso** (*Reusability*) permette di usare un asset in più di un sistema.
- L'**analizzabilità** (*Analysability*) permette di comprendere i difetti e valutare l'impatto dei cambiamenti.
- La **modificabilità** (*Modifiability*) permette di effettuare cambiamenti preservando la qualità richiesta.
- La **testabilità** (*Testability*) permette di stabilire criteri e verificare se sono soddisfatti.

Flessibilità

La **Flexibility** è il grado in cui un prodotto può essere adattato a cambiamenti nei requisiti, nei contesti d'uso o nell'ambiente operativo. Si articola in:

- L'**adattabilità** (*Adaptability*) misura quanto il sistema può essere adattato o trasferito efficacemente a diversi hardware, software o ambienti operativi/d'uso.
- La **scalabilità** (*Scalability*) misura quanto il sistema può gestire carichi crescenti o decrescenti o adattarne la capacità alla variabilità.
- L'**installabilità** (*Installability*) misura l'efficacia e l'efficienza con cui il prodotto può essere installato e/o disinstallato con successo.
- La **sostituibilità** (*Replaceability*) misura quanto il prodotto può sostituire un altro prodotto software specificato per lo stesso scopo nello stesso ambiente.

Sicurezza (Safety)

La **Safety** è il grado in cui un prodotto evita uno stato in cui sono messe in pericolo vita umana, salute, proprietà o ambiente. Include, tra le altre:

- Il **fail safe** misura quanto il prodotto può porsi automaticamente in una modalità operativa sicura o tornare a una condizione sicura in caso di guasto.
- L'**identificazione dei rischi** (*Risk identification*) misura quanto il prodotto può individuare eventi o operazioni che possono portare a rischi inaccettabili.
- La **segnalazione dei pericoli** (*Hazard warning*) misura quanto il sistema fornisce avvisi su rischi inaccettabili a operazioni o controlli interni, così che possano reagire in tempo utile.

La qualità in uso

Il **modello di qualità in uso** (ISO/IEC 25019) è composto da **3 caratteristiche**, ulteriormente suddivise in sotto-caratteristiche, che possono influenzare gli stakeholder quando i prodotti sono usati in un contesto d'uso specificato. Misura il grado in cui un prodotto può essere usato da utenti specifici per soddisfare i propri

bisogni e raggiungere obiettivi specifici con **efficacia, efficienza, assenza di rischio e soddisfazione** in contesti d'uso specifici.

Usabilità

L'**Usability** misura in che misura gli utenti possono raggiungere i propri obiettivi in modo efficiente e soddisfacente usando il sistema:

- L'**efficacia** (*Effectiveness*) misura quanto bene gli utenti completano i compiti previsti usando il sistema.
- L'**efficienza** (*Efficiency*) riguarda le risorse richieste, ad esempio tempo e sforzo, per svolgere i compiti.
- La **soddisfazione** (*Satisfaction*) riguarda il comfort e l'esperienza positiva dell'utente durante l'uso del sistema.

Sicurezza

La **Safety** valuta la capacità del sistema di prevenire danni a persone, ambiente e interessi commerciali:

- Il **danno commerciale** (*Commercial Damage*) valuta quanto il sistema previene perdite finanziarie o danni all'attività.
- La **salute e sicurezza degli operatori** (*Operator Health and Safety*) misura quanto il sistema protegge gli utenti da rischi per la salute o per la sicurezza durante l'uso.
- La **salute e sicurezza pubblica** (*Public Health and Safety*) riguarda la prevenzione di rischi o danni per il pubblico generale dall'uso o dall'esercizio del sistema.
- Il **danno ambientale** (*Environmental Harm*) riguarda la capacità del sistema di evitare o minimizzare gli impatti negativi sull'ambiente.

Flessibilità

La **Flexibility** riguarda la capacità del sistema di adattarsi e operare efficacemente in contesti o ambienti diversi:

- La **conformità al contesto** (*Context Conformity*) è la capacità del sistema di adattarsi ai requisiti e ai vincoli specifici di contesti diversi.
- L'**estensibilità di contesto** (*Context Extensibility*) è il potenziale del sistema di espandersi o adattarsi a ambienti nuovi o in cambiamento senza modifiche significative.
- L'**accessibilità** (*Accessibility*) misura quanto efficacemente il sistema può essere usato da tutte le persone, incluse quelle con disabilità, in ambienti diversificati.

Misurare la qualità del software

Valutare un attributo di qualità

Un attributo di qualità individua una **preoccupazione** (*concern*), come l'efficienza prestazionale o la manutenibilità, ma non è ancora una misura. Per valutarlo servono cinque elementi: un **obiettivo di valutazione** (quale decisione dobbiamo prendere), una **misura** (quale proprietà osserveremo e come), un **contesto** (quali compiti, utenti, carico di lavoro e ambiente), un **criterio di accettazione** (quali risultati considereremo accettabili) e le **evidenze** (le osservazioni raccolte con una procedura documentata).

La distinzione fondamentale è che **la misurazione produce un risultato, mentre la valutazione interpreta quel risultato rispetto a criteri concordati**. Ne deriva una regola pratica: i criteri vanno concordati **prima** di raccogliere i risultati, altrimenti l'interpretazione risulta arbitraria.

Un requisito di qualità misurabile: prestazioni di ricerca delle aule

Il bisogno degli stakeholder è che gli studenti trovino le aule disponibili senza attese eccessive. Le condizioni di valutazione sono: 1.000 record di aule, 100 utenti simultanei simulati, un carico di ricerca fisso su un setup di riferimento documentato e un periodo di misurazione di 10 minuti dopo un warm-up di 2 minuti.

I criteri di accettazione sono due:

- Almeno il **95% delle richieste di ricerca valide** deve restituire risultati corretti entro **2 secondi**.
- Al massimo lo **0,1% delle richieste valide** può fallire.

Il tempo viene misurato sul client di test, dall'invio della richiesta alla ricezione della risposta completa. Un dettaglio metodologico importante: errori, risposte errate e timeout contano come **fallimenti**, quindi non possono essere contati come risposte rapide andate a buon fine.

Risultati della valutazione

I risultati illustrativi di una singola esecuzione del test sono:

Osservazione	Richieste
Risposta corretta entro 2 secondi	1.940
Risposta corretta dopo 2 secondi	50
Richiesta fallita	10
Totale	2.000

Dai dati si ricava la valutazione. Le risposte riuscite tempestive sono 1.940 su 2.000, cioè il 97%, quindi il criterio 95% è soddisfatto. Le richieste fallite sono invece 10 su 2.000, cioè lo 0,5%, quindi il criterio 0,1% **non** è soddisfatto. Di conseguenza la versione testata **non soddisfa i criteri di accettazione combinati**: occorre indagare i fallimenti, implementare miglioramenti e ripetere la valutazione in condizioni confrontabili. Questi risultati descrivono questa specifica versione nelle condizioni testate, non il sistema in generale.

Evidenze e limiti della valutazione

Poiché preoccupazioni di qualità diverse richiedono evidenze diverse, non esiste una misura universale della qualità. Ad esempio:

- La **manutenibilità** si evidenzia tramite analisi delle dipendenze, design review e sforzo richiesto per compiti di cambiamento definiti.
- L'**affidabilità** si evidenzia tramite osservazioni di guasti, test di interruzione e risultati di ripristino.
- L'**usabilità** si evidenzia tramite completamento dei compiti, tempi e soddisfazione con utenti rappresentativi.

Le evidenze vanno interpretate con cautela per quattro motivi: una misura di solito coglie solo una parte di un attributo; i risultati dipendono dalla procedura e dal contesto di valutazione; misure ripetute aiutano a rivelare la variabilità; e una copertura dei test elevata, da sola, **non stabilisce la correttezza**. Per questo motivo una valutazione della qualità dovrebbe sempre riportare criteri, risultati e limiti, insieme alla decisione che essi supportano.

Lezione 3 e 4 — Requirements Engineering: Elicitation and Analysis

Info

Citazione chiave

«La parte più difficile della costruzione di un sistema software è decidere precisamente cosa costruire» — Fred P. Brooks. Nessun errore, se fatto in questa fase, è più difficile da correggere in seguito.

Il costo di correzione di un errore cresce con la fase del ciclo di vita in cui viene scoperto; gli errori di requisiti sono quindi i più costosi.

<Immagine | Cost of errors through project lifecycle>

Software Requirements

Definition

Requisito

Un **requisito** è una descrizione di ciò che il sistema deve fare: **servizi** che il sistema deve fornire ai propri utenti e **vincoli operativi** a cui è sottoposto.

Livelli di requisiti

Il termine «requisito» è usato in modo incoerente nell'industria; distingue due livelli di astrazione.

- **Requisiti utente** (*user requirements*): descrizione astratta e ad alto livello di un servizio o vincolo, nella prospettiva dell'utente finale.
- **Requisiti di sistema** (*system requirements*): definizione dettagliata e formale di una funzione del sistema, nella prospettiva del sistema da costruire.

L'ambiguità tra i due livelli è inevitabile perché i requisiti svolgono una doppia funzione contrattuale: i **requisiti utente** possono essere la base di un bando di gara, mentre i **requisiti di sistema** — formulati dal contraente vincitore e validati dal committente — entrano nel contratto finale e sono **vincolanti**.

I requisiti esistono a livelli di astrazione crescente, dal business al dettaglio:

- **Business goal**: obiettivo aziendale che motiva il progetto (es. ridurre le visite mancate).
- **User requirement**: servizio richiesto dall'utente.
- **System requirement**: comportamento preciso del sistema.
- **Quality / constraint**: proprietà quantitativa o vincolo sul comportamento.

Tipi di requisiti

- **Requisiti funzionali** (*functional requirements*): cosa il sistema deve fare.
- **Requisiti di qualità** (*quality requirements*): quanto bene il sistema deve svolgere le sue funzioni (prestazioni, affidabilità, sicurezza, manutenibilità, ...).
- **Vincoli** (*constraints*): tecnologici, organizzativi, legali/regolatori, di processo.
- **Requisiti di dominio** (*domain requirements*): derivanti dal contesto o settore d'uso del software; possono essere funzionali o non e vincolano sia il progetto sia il processo di sviluppo (es. uno standard di sicurezza medica).

I requisiti di qualità e i vincoli sono spesso chiamati collettivamente **requisiti non funzionali**.

Requisiti non funzionali: classificazione

- **Requisiti di prodotto** (*product requirements*): caratteristiche richieste al prodotto — usabilità, efficienza (prestazioni, spazio), dipendenza (affidabilità, sicurezza).
- **Requisiti organizzativi** (*organizational requirements*): derivanti dall'organizzazione di committente e sviluppatori — ambientali, operativi, di sviluppo, contabili.
- **Requisiti esterni** (*external requirements*): derivanti da fonti esterne — regolatori, legislativi, etici, di sicurezza.

I non-funzionali non riguardano servizi specifici ma **caratteristiche del sistema nel suo insieme**; spesso sono più critici dei funzionali: un requisito funzionale subottimale si aggira, mentre il fallimento di un

non-funzionale può rendere il sistema inutilizzabile o non schierabile (es. per mancata conformità al GDPR). La distinzione funzionale/non-funzionale **non è netta**: un non-funzionale, sviluppato in dettaglio, genera requisiti funzionali (es. «solo utenti autorizzati» genera il login). I requisiti non sono indipendenti: uno spesso genera o vincola altri.

Warning

Testabilità

I non-funzionali di sistema devono includere **indicatori quantitativi** quando possibile: «il sistema deve essere affidabile» non è verificabile, «uptime mensile del 99,9%» lo è.

Metriche tipiche per i requisiti non funzionali:

Proprietà	Metriche
Prestazioni	Operazioni/secondo, tempo di risposta a utente/evento, frequenza di refresh dello schermo
Dimensione	Megabyte
Facilità d'uso	Tempo di formazione richiesto, tasso di errori utente, numero di richieste di supporto
Affidabilità	Tempo medio a guasto, tasso di disponibilità (uptime)
Robustezza	Tempo di ripristino dopo guasto, probabilità di perdita dati al guasto

Proprietà di un buon requisito

- **Chiaro e comprensibile**: soprattutto per i requisiti utente.
- **Non ambiguo**: l'ambiguità porta a dispute col committente.
- **Completo e consistente**: senza conflitti reciproci.
- **Necessario**: se ne deve capire la ragione d'essere.
- **Fattibile**: implementabile realisticamente.
- **Tracciabile**: si deve sapere da dove proviene e cosa dipende da esso.
- **Verificabile**: deve essere possibile stabilire senza ambiguità se il sistema lo soddisfa.

Example

I demoni dell'ambiguità

Un requisito apparentemente chiaro nasconde domande non risposte (es. «segnale per i bambini» non definisce chi è un bambino né quando la regola si applica). Analogamente, un requisito di avvio/stop di un servizio remoto non specifica comportamento su stati già attivi, riavvii, cadute di connessione, errori e retry: i requisiti di sistema devono contenere dettaglio sufficiente a essere non ambigui e verificabili.

Il processo di Requirements Engineering

Definition

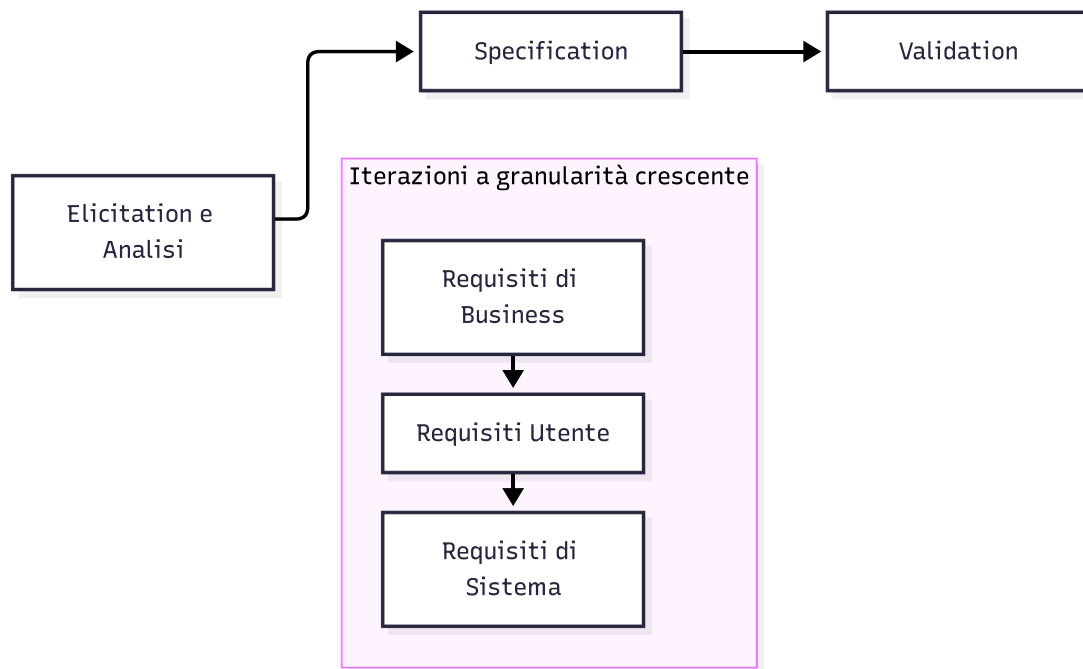
Requirements Engineering (RE)

Sottoarea dell'ingegneria del software che fornisce metodi, tecniche e strumenti per comprendere e documentare cosa un sistema software deve fare.

Tre attività chiave:

1. **Elicitation e analisi**: scoprire i requisiti interagendo con gli stakeholder.
2. **Specification**: convertire i requisiti in una forma standardizzata.
3. **Validation**: verificare che i requisiti definiscano davvero il sistema voluto dal committente.

In pratica il processo **non è lineare**: le attività sono interleaveate in un processo iterativo su livelli di granularità crescente — prima requisiti di **business**, poi **utente**, infine **di sistema** — e ogni iterazione include elicitation/analisi, specification e validation (con feasibility study, prototyping e review come tecniche di supporto).



Requirements Elicitation and Analysis

L'elicitazione è considerata la parte **più critica** del processo RE e coinvolge la collaborazione tra **stakeholder**.

Definition

Stakeholder

Persone o gruppi affetti in qualche modo dal progetto: utenti finali (gruppo spesso eterogeneo), utilizzatori degli output del software, management e dirigenti del cliente, personale tecnico del cliente.

Sfide dell'elicitazione

- Gli stakeholder **non sanno cosa vogliono**, se non nei termini più generali, e possono avanzare richieste irrealistiche.
- Gli stakeholder sono **esperti del proprio dominio**: usano gergo e conoscenza implicita, dando per scontati dettagli che non lo sono.
- Stakeholder diversi esprimono **bisogni diversi in modi diversi**: occorre gestire comunanze e conflitti.
- **Fattori politici** possono influenzare il processo (es. manager che chiedono requisiti per accrescere la propria influenza).
- Le priorità sono contrastanti e stakeholder che si sentono ignorati possono **sabotare** il processo.
- I requisiti **cambiano** durante l'elicitazione, anche per nuovi stakeholder emersi in corsa.

Processo di elicitation e analisi

1. **Discovery e comprensione**: interazione con gli stakeholder per scoprire i requisiti.
2. **Classificazione e organizzazione**: raggruppamento dei requisiti correlati.
3. **Prioritizzazione e negoziazione**: risoluzione dei conflitti tra bisogni contrastanti.
4. **Documentazione**: tracciamento dei requisiti per l'iterazione successiva.

Tecniche di elicitation

Interviste con gli stakeholder

- **Interviste chiuse** (*closed*): il risponditore risponde a una lista predefinita di domande.
- **Interviste aperte** (*open*): nessuna agenda predefinita.

- In pratica si combinano: le interviste totalmente aperte raramente funzionano; conviene partire da domande predisposte che aprono discussioni meno strutturate.

Limiti e accorgimenti:

- Le persone parlano volentieri del proprio lavoro, ma occorre **non sprecare il loro tempo** e minimizzare l'impatto sul loro lavoro.
- Lo **gergo** e le conoscenze implicite rendono ambigue le risposte.
- Ogni stakeholder ha una **visione parziale o distorta** del lavoro dei colleghi di altre aree.
- Dinamiche di potere possono rendere gli stakeholder **riluttanti** a discutere requisiti e vincoli organizzativi.

Etnografia

- Il software esiste in un **ambiente sociale e organizzativo** che genera o vincola i requisiti; spesso i processi realmente usati **differiscono** da quelli formali dichiarati.
- È una tecnica **osservativa**: l'ingegnere dei requisiti si immerge nell'ambiente di lavoro degli utenti finali e osserva il lavoro quotidiano, annotando i compiti effettivi e i partecipanti coinvolti.

È particolarmente efficace per scoprire:

- Requisiti derivanti da **come le persone lavorano davvero**, non da come i processi formali dicono che dovrebbero lavorare.
- Requisiti derivanti da **cooperazione e consapevolezza** delle attività altrui (es. verificare con i colleghi se un ordine è già stato consegnato).

Personas

Definition

Persona

Archetipo **ipotetico** di utenti reali: non è una persona reale né completamente inventata, ma viene **scoperta** come sottoprodotto dell'elicitazione e definita con rigore e precisione. È basata su evidenze, non su stereotipi.

- Motivazione: gli utenti reali del software sono **diversi dagli sviluppatori**; le personas promuovono **empatia** e comprensione completa degli utenti.
- Sono essenziali quando **non ci sono stakeholder da intervistare** (es. software off-the-shelf per il grande pubblico).
- Non esiste una rappresentazione standard; elementi comuni:
 - **Personalizzazione**: nome, età, breve biografia.
 - **Informazioni lavorative**: qualifica e ruolo.
 - **Educazione** ed esperienza.
 - **Obiettivi**: interesse nell'uso del software.
 - **Frustrazioni / pain points**: criticità che il software può risolvere.

Da una persona possono **emergere nuovi requisiti** che gli stakeholder non avevano espresso (es. utenti non tecnologici che necessitano di prenotazione telefonica e reminder umani).

Stories

- È più facile rapportarsi a **esempi reali** che ad astrazioni: gli stakeholder descrivono bene *come fanno le cose*, non *come devono essere i requisiti*.
- Una **storia** è una descrizione narrativa, ad alto livello, di come il sistema viene usato per un compito particolare: cosa fa l'utente, quali informazioni usa, quale output serve.

Low-fidelity mockups

Definition

Wireframe

Schizzo semplificato dell'interfaccia utente del sistema; il focus è su **funzionalità e flusso**, non sull'estetica.

Benefici:

- **Facilitano la comprensione**: punto di partenza per la discussione e per una comunicazione chiara delle idee iniziali.
- **Aiutano a scoprire nuovi requisiti**: è più facile ragionare su interfacce concrete che su affermazioni astratte.
- **Engagement e feedback iterativo** senza grandi investimenti di design.
- **Fondamenta per lo sviluppo**: base per passare ai design a maggiore fedeltà e infine allo sviluppo.

Strumenti: carta e matita per gli schizzi iniziali; strumenti digitali commerciali (Balsamiq, Figma) o open-source (PenPot) per creare e condividere wireframe elettronici.

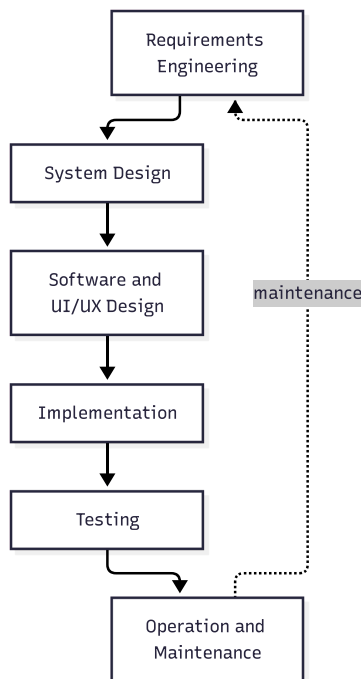
Collocazione nel ciclo di vita

Nel ciclo di vita del software, la RE precede System Design e Software/UI-UX Design: i requisiti sono raccolti tramite interviste, personas e specificati con storie/scenari, use case, linguaggio naturale, modelli di dominio e mock-up; l'architettura alloca poi sottosistemi e requisiti su risorse hardware e software.

Lezione 5 e 6 — Requirements Engineering: Specification

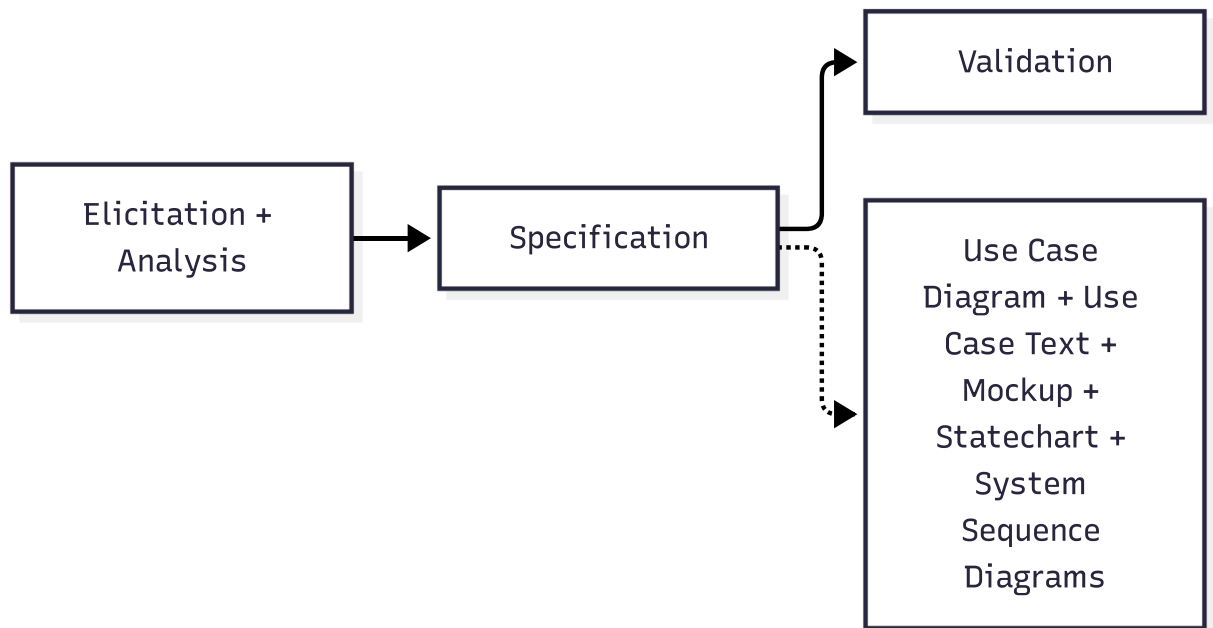
Il ciclo di vita del software e il posto della RE

- Il **ciclo di vita del software** è composto dalle fasi: Requirements Engineering → System Design → Software and UI/UX Design → Implementation → Testing → Operation and Maintenance.
- Nella **Requirements Engineering** i requisiti sono raccolti tramite interviste con stakeholder, personas, storie e scenari; sono specificati con Use Cases, linguaggio naturale, domain model e mock-up.
- Nel **System Design** si definisce l'architettura del sistema: i requisiti sono allocati a sotto-sistemi software, e i sotto-sistemi a risorse hardware; si usano architectural pattern.
- Nel **Software and UI/UX Design** si definiscono gli oggetti necessari a realizzare ogni sottosistema; si usano design pattern, usability engineering e wireframing ad alta fedeltà.
- Nell'**Implementation** ogni sottosistema viene implementato (codice sorgente e altri artefatti), con attenzione a clean code, framework/ORM e qualità del software.
- Nel **Testing** si verifica che il software soddisfi il cliente: code inspection, functional testing (unit, integration, system) e usability testing.
- In **Operation and Maintenance** il sistema entra in uso pratico; la manutenzione serve a correggere errori non scoperti prima e ad adattare il software a cambiamenti nei requisiti o nell'ambiente.



Il processo di Requirement Engineering

- Il processo ha tre fasi: **Elicitation** + **Analysis** → **Specification** → **Validation**.



Requirements Specification

- La **Requirement Specification** è il processo di scrivere i requisiti utente e di sistema in un **requirements document**.
- Gli **user requirements** devono essere comprensibili a utenti finali e committenti senza background tecnico.
- I **system requirements** sono requisiti più dettagliati e possono includere informazioni più tecniche.
- I requisiti possono far parte di un contratto per lo sviluppo del sistema, quindi devono essere **completi e dettagliati** il più possibile.

Requirements e design

- In principio i **requisiti** dicono *cosa* il sistema deve fare e il **design** *come* lo fa; in pratica sono **inseparabili**:
 - I requisiti possono essere strutturati e organizzati sulla base di un'architettura di alto livello del sistema.
 - Il sistema può interagire con altri sistemi che generano requisiti di design.
 - L'uso di un'architettura specifica per soddisfare requisiti non-funzionali può essere un requisito di dominio.

Approcci alla specifica

- **Natural Language**: requisiti come frasi numerate in linguaggio naturale; ogni frase esprime un requisito.
- **Structured Natural Language**: si usa una form o template standardizzato.
- **Semi-formal notations and models**: UML Use Case Diagram e altri domain model, tipicamente integrati da annotazioni in linguaggio naturale.
- **Formal Specification**: notazioni basate su concetti matematici come macchine a stati finite e infinite, logiche temporali.

Info Scelta dell'approccio per dominio

- Nei sistemi **safety-critical** si usano comunemente specifiche formali e linguaggio naturale strutturato.
- In un'app **todo list** si può usare linguaggio naturale non strutturato.
- In un **sistema informativo medio-grande** le notazioni semi-formali sono un buon compromesso.

Specifica in linguaggio naturale (NL)

- I requisiti sono scritti come frasi in linguaggio naturale, opzionalmente integrati da diagrammi e tabelle.

- È usata perché è **espressiva**, **intuitiva** e **universale**: i requisiti risultano comprensibili a utenti e committenti.
- Può esprimere requisiti sia **funzionali** che **non-funzionali**.

Linee guida per la NL

- Definire un formato “standard” e usarlo per tutti i requisiti, es.: `<part of the system> shall <requirement> (<rationale>)`.
 - **shall** per requisiti obbligatori, **should** per requisiti desiderabili.
- Usare l’evidenziatura del testo per identificare le parti chiave del requisito.
- Evitare il gergo informatico.
- Includere la spiegazione (**rationale**) del perché un requisito è necessario.

Problemi del linguaggio naturale

- **Lack of clarity**: la precisione è difficile senza rendere il documento difficile da leggere.
- **Requirements confusion**: requisiti funzionali e non-funzionali tendono a essere mescolati.
- **Requirements amalgamation**: più requisiti diversi possono essere espressi insieme nella stessa frase.

Specifiche strutturate

- La **Structured Specification** è un approccio in cui la libertà del redattore è limitata e si impone un modo standardizzato di scrivere i requisiti.
- Funziona bene per certi tipi di requisiti (es. sistemi embedded di controllo) ma a volte è troppo rigida per i requisiti di sistemi aziendali (business system).

Example

Esempio di form strutturata (Insulin Pump/Control Software/SRS/3.3.2)

Ogni requisito è descritto con campi fissi:

Campo	Contenuto
Function	Compute insulin dose: Safe sugar level
Description	Computa la dose di insulina quando il livello di zucchero misurato è tra 3 e 7 unità
Inputs	Lettura corrente dello zucchero (r_2), le due letture precedenti (r_0 e r_1)
Source	Lettura corrente dal sensore; le altre letture dalla memoria
Outputs	CompDose — la dose di insulina da somministrare
Action	CompDose è zero se il livello è stabile o in caduta, o se cresce ma il tasso di crescita diminuisce; se livello e tasso di crescita sono entrambi crescenti, CompDose = differenza tra livello corrente e precedente divisa 4, arrotondata (se l’arrotondamento dà zero, si usa la dose minima erogabile)
Requires	Le due letture precedenti, per calcolare il tasso di variazione
Precondition	Il serbatoio di insulina contiene almeno la dose singola massima consentita
Postcondition	In memoria r_0 è sostituito da r_1 , poi r_1 da r_2
Side effects	Nessuno

Use Case Diagrams

Use Cases

- Gli **Use Cases** descrivono le interazioni tra utenti e sistema usando un modello grafico e testo strutturato in linguaggio naturale.

- Sono una parte chiave dell'**UML** (Unified Modelling Language) e rappresentano l'insieme dei **requisiti funzionali** di un sistema.
- Identificano:
 - **Attori (Actors)** — categorie di utenti del sistema (non necessariamente umani).
 - **Use Cases** — obiettivi (goal) di un attore, offerti dal sistema.
- Informazioni aggiuntive sulle interazioni possono essere fornite da descrizioni testuali strutturate o da modelli semi-formali (es. Sequence Diagram UML o Statechart).
- Scopo: supportare la comunicazione col cliente per definire le funzionalità del sistema; devono essere il più semplici possibile.
- **Non definiscono come il sistema è implementato, ma cosa il sistema deve fare**, dal punto di vista degli utenti (il sistema è una **black box**).
- Vengono spesso documentati con un **Use Case Diagram (UCD)** di alto livello.

Attori

- Gli attori sono rappresentati con **stick figure** e hanno un nome unico.
- Rappresentano entità **esterne** che interagiscono con il **System Under Development (SUD)**:
 - Classi di utenti (umani).
 - Altri sistemi.
 - Ambiente fisico.
- Gli attori sono più **coarse-grained** delle Personas: un attore può essere associato a più Personas.

Euristiche per identificare gli attori

- Quali gruppi di utenti sono supportati dal SUD nel loro lavoro?
- Quali gruppi eseguono le funzionalità principali offerte dal SUD?
- Quali gruppi eseguono le funzioni secondarie (es. amministrazione)?
- Il SUD interagisce con sistemi o software esterni? Ogni sistema esterno con cui il SUD interagisce è un attore.
- Un attore non corrisponde a una singola entità ma a una **classe di utenti con lo stesso ruolo**; uno stesso utente può avere ruoli diversi nello stesso sistema (es. l'amministratore di un sito può anche visitarne le pagine come utente non autenticato).

Use Cases

- I use case sono rappresentati come **ellissi nominate**.
- **Un Use Case rappresenta un obiettivo che un attore vuole raggiungere interagendo col sistema**, fornendo un **beneficio** o **utilità** all'attore.
- Il nome è tipicamente un **active verb phrase** che descrive il goal dell'attore: *Withdraw Money, Submit Incident Report, Borrow Book, Generate Monthly Report*.
- I use case modellano i **requisiti funzionali**.

Use case e scenari

- Un use case astrae molti possibili **scenari** (sequenze di azioni) per lo stesso obiettivo.
- Uno **scenario** è un'istanza di un use case; il use case rappresenta una classe di scenari che usano la stessa funzionalità.

System Boundary

- Prima di identificare attori e use case, definire chiaramente il **SUD** (System Under Development):
 - Gli attori sono **esterni** al SUD.
 - I use case descrivono comportamenti **forniti dal SUD**.
 - Spostare il **system boundary** può cambiare sia attori sia use case.
- Ciò che va sviluppato è **dentro** il SUD; ciò che è già disponibile **non** lo è.

Relazioni nel UCD

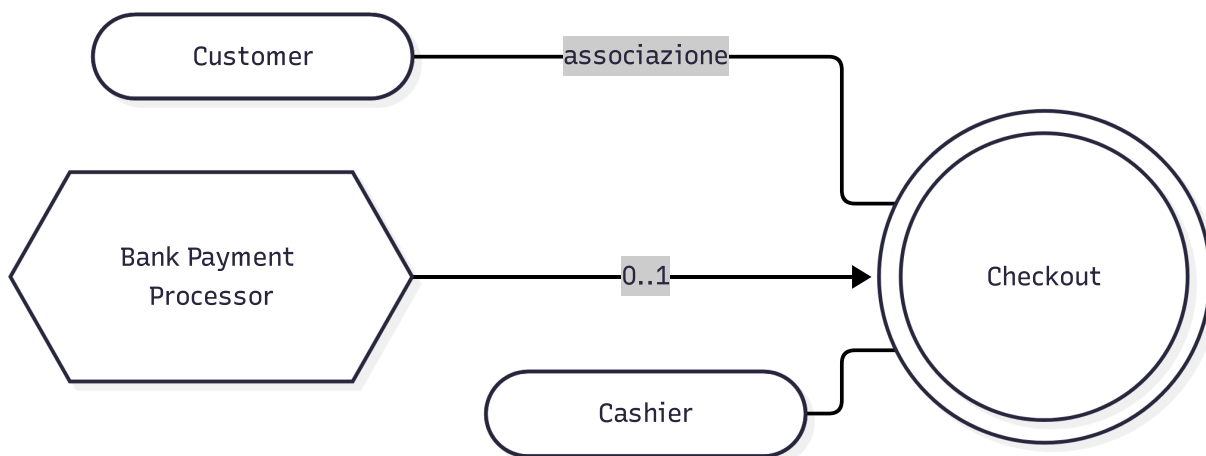
Associazioni

- Un'**associazione** indica che l'attore partecipa al Use Case interagendo col sistema.
- Graficamente è una **linea** che collega attore e use case.

Attori secondari

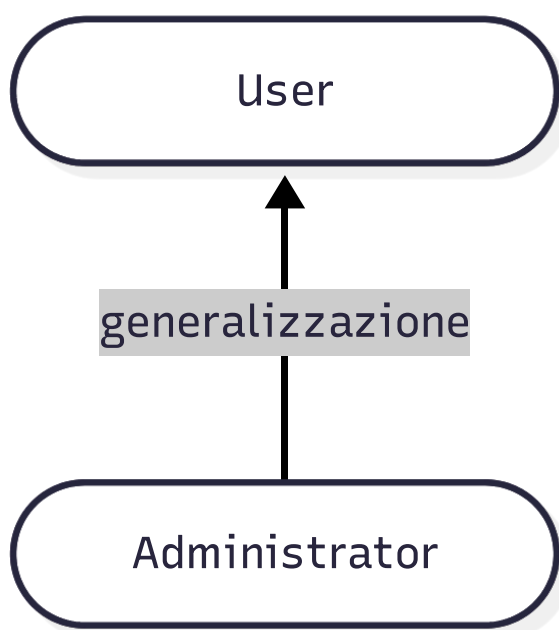
- Un use case può essere associato a più attori:
 - **Primary Actor**: l'attore il cui obiettivo è soddisfatto dal use case.

- **Supporting/Secondary Actor:** entità esterna da cui il sistema ha bisogno di un servizio durante l'esecuzione del use case.
- La partecipazione di un attore secondario può essere opzionale (**cardinalità** 0..1, es. un Bank Payment Processor coinvolto solo se il cliente paga con carta).
- Il UCD **non** specifica *come* i diversi attori sono coinvolti né le loro responsabilità: queste si specificano con descrizioni aggiuntive.



Generalizzazione di attori

- La **generalizzazione tra attori** si applica quando un attore è sotto-tipo di un altro.
- Stesso concetto e notazione dei Class Diagram UML: freccia con **testa vuota** dal figlio al genitore.
- L'attore specializzato può eseguire tutti i use case che il genitore può eseguire.



Errori comuni (Rookie Mistakes)

Warning

Errori comuni nella modellazione dei use case

- Ogni use case deve dare un **beneficio** all'attore (aiutarlo a completare il suo lavoro o raggiungere un obiettivo): i nomi dei use case contengono tipicamente un verbo, i nomi degli attori un sostantivo.
- Due attori associati allo stesso use case (cardinalità non nulla) sono **coinvolti e devono collaborare in ogni istanza** (scenario) del use case: **non** significa che ciascuno può eseguirlo indipendentemente.
- Attenzione alle generalizzazioni improprie: ogni attore deve avere i propri use case; l'attore specializzato eredita già tutti i use case del genitore — se non ha use case propri, la generalizzazione è inutile o mancano use case.

- Il diagramma non deve diventare troppo complesso: relazioni tra use case e generalizzazioni vanno usate con moderazione, la modellazione deve restare a un alto grado di astrazione e ordinata (evitare linee che si incrociano). **Un diagramma complesso indica un cattivo analista.**

Livello giusto del use case

- Un use case deve corrispondere a un **user goal**, non a una singola operazione di UI.
- Troppo piccoli: *Enter password*, *Click Checkout*, *Select payment method*.
- Obiettivi appropriati: *Authenticate User*, *Purchase Product*, *Submit Incident Report*.
- Domanda guida: **“What is the actor trying to achieve?”** — se completare il use case non fornisce valore significativo all’attore, è troppo fine-grained.

Specifica dettagliata dei Use Cases

- Il UCD dà una visione di insieme di alto livello dei requisiti funzionali, ma **non è abbastanza dettagliato** per stabilire i requisiti di sistema.
- Per **ogni** UC del diagramma serve una specifica dettagliata, che renda il comportamento richiesto chiaro e **verificabile**.
- Vanno descritti il **Main Success Scenario** e **tutti gli scenari alternativi e di fallimento rilevanti**.

Contenuto di una descrizione testuale

- Descrizione di ciò che sistema e utenti si aspettano quando il use case inizia.
- Descrizione del flusso normale di eventi (**main scenario**).
- Descrizione di ciò che può causare errori e come gestirli.
- Descrizione dello stato del sistema al termine del use case.

Formati dei use case

- **Brief**: sintetico riassunto in un paragrafo, di solito del solo main success scenario.
- **Casual**: formato informale a paragrafi che coprono vari scenari.
- **Fully-dressed**: tutti i passi e le variazioni descritti in dettaglio, con sezioni di supporto (precondizioni, garanzie di successo).
- Brief e Casual si usano nelle fasi iniziali della specifica, per capire rapidamente soggetto e perimetro; i Fully-dressed si sviluppano dopo, come base per un **contratto** e per specificare in dettaglio il comportamento del sistema.

Template di Cockburn (semplificato)

- Per le descrizioni fully-dressed esistono diversi formati proposti; questo è basato sul template di **Alistair Cockburn**.

USE CASE #X	Name of the Use Case
Goal in Context	Descrizione dell’obiettivo del UC
Preconditions	Tutte le condizioni che devono valere per avviare il UC
Success End Condition	Stato del sistema se il UC ha successo
Failed End Condition	Stato del sistema se il UC fallisce
Primary Actor	Attore primario del UC
Trigger	Azione dell’attore primario che avvia il UC
Main Scenario	Tabella a step: Actor 1 Actor n System
Extensions	Per ogni estensione: tabella a step riferita al main scenario
Open Issues	Aspetti ancora da chiarire; alla consegna del documento deve essere vuota

- Il **main scenario** è la sequenza di azioni che si verifica quando tutto procede senza intoppi.

- Ma ci possono essere modi diversi di eseguire un use case (es. autenticazione con PIN o con impronta) o errori in qualche punto: descrivere anche queste sequenze alternative è fondamentale per il comportamento funzionale, usando le **Extensions** (tipicamente il testo nelle estensioni è molto più abbondante che nel main scenario).

Scrivere un'estensione

Ogni estensione identifica quattro elementi:

1. **Where?** — riferimento allo step del main scenario.
2. **When?** — condizione che innesca l'estensione.
3. **What happens?** — sequenza alternativa.
4. **Then?** — terminare, riuscire diversamente, o **ritornare a uno step del main scenario**.

Example

Estensione tipica (fondi insufficienti, riferita a uno step x)

- *xa* — Il sistema informa il cliente che l'importo richiesto non è disponibile.
 - Il sistema chiede un importo diverso.
 - Si riprende dallo step precedente del main scenario.
- Scenari di fallimento ed errori da dettagliare come estensioni: PIN errato, saldo insufficiente del cliente, riserve di cassa insufficienti dell'ATM, carta rubata, carta illeggibile.
 - Variazioni possibili (esecuzione diversa, non errore): autenticazione con impronta o riconoscimento facciale invece del PIN, stampa opzionale della ricevuta.

Requirements Validation

- La **Requirements Validation** dimostra che i requisiti definiscono il sistema che il cliente vuole davvero.
- Gli errori nei requisiti hanno costi alti: correggere un errore di requisito dopo la consegna può costare fino a **100 volte** la correzione di un errore di implementazione.

Requirement checking

- **Validity**: il sistema fornisce le funzioni che supportano al meglio i bisogni del cliente?
- **Consistency**: ci sono requisiti in conflitto?
- **Completeness**: tutte le funzioni richieste dal cliente sono incluse?
- **Realism**: i requisiti sono implementabili con il budget e la tecnologia disponibili?
- **Verifiability**: i requisiti sono controllabili?

Tecniche di validazione

- **Requirements reviews**: analisi manuale sistematica dei requisiti.
- **Prototyping**: uso di un modello eseguibile semplificato del sistema per verificare i requisiti; anche *visual prototyping* con mockup/wireframe.
- **Test-case generation**: sviluppo di test per i requisiti, per verificarne la testabilità.

Requirements reviews

- Vanno tenute revisioni regolari mentre i requisiti vengono formulati.
- Devono coinvolgere sia lo staff del cliente sia quello dell'appaltatore.
- Possono essere formali (su documenti completati) o informali; una buona comunicazione tra sviluppatori, clienti e utenti risolve i problemi in fase iniziale.

Review checks

- **Verifiability**: il requisito è realisticamente testabile?
- **Comprehensibility**: il requisito è compreso correttamente?
- **Traceability**: l'origine del requisito è dichiarata chiaramente?
- **Adaptability**: il requisito può essere cambiato senza grande impatto sugli altri requisiti?

Lezione — UML: Class Diagram e Sequence Diagram

Il Class Diagram e i suoi limiti

Definition Class Diagram

Strumento principale dell'**UML** (*Unified Modeling Language*), descrive le idee e la struttura del software a vari livelli di dettaglio e in diverse fasi del ciclo di vita: dalla bozza iniziale dell'architettura fino alla definizione concreta, da cui generare scheletri di codice (es. Java).

- **Struttura statica e astratta:** descrive solo quali classi esistono, i loro **attributi** e **metodi**, e le relazioni strutturali tra loro.
- **Mancanza di dinamica:** non specifica la logica interna dei metodi né **come le classi e gli oggetti collaborano nel tempo** per realizzare una funzionalità o un caso d'uso (*use case*).
- Dalla generazione di codice da un Class Diagram si ottengono **metodi vuoti**, privi di implementazione.

I Sequence Diagram

Definition Sequence Diagram

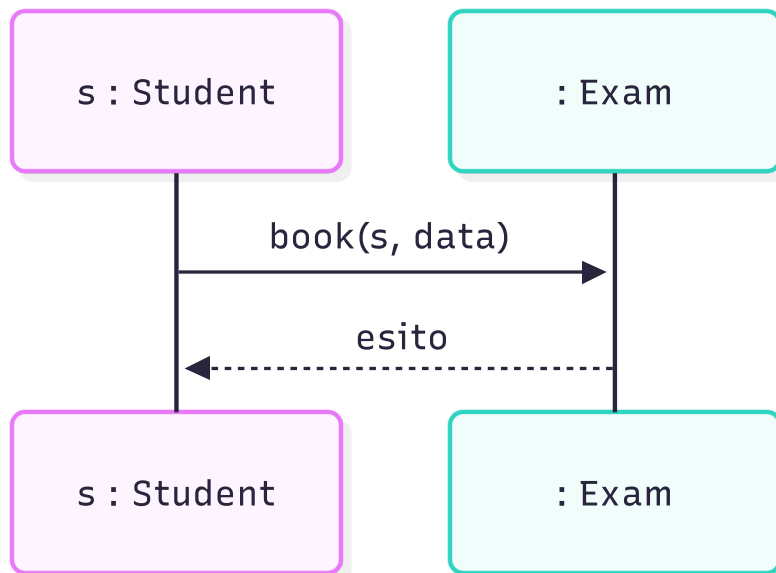
Diagramma UML che descrive il funzionamento **dinamico** del sistema e l'interazione tra oggetti nel tempo; è il secondo diagramma per importanza in UML.

Confronto con il Class Diagram

- **Molteplicità:** nel modello di un sistema ci sono pochissimi Class Diagram (spesso uno per sistema o modulo), ma **molte** **Sequence Diagram**, uno per ogni funzionalità o scenario dei casi d'uso.
- **Verso di lettura:** il Class Diagram **non ha un verso di lettura** prestabilito; il Sequence Diagram ha un rigoroso **senso di lettura temporale**.
- **Parte alta:** contiene gli oggetti/istanze che collaborano.
- **Asse verticale:** rappresenta il trascorrere del tempo, dall'alto verso il basso.
- **Asse orizzontale:** rappresenta lo scambio di **messaggi** (chiamate di metodo) tra gli oggetti.

Sintassi di base

- **Oggetti/istanze:** sintassi `nomeOggetto : NomeClasse` (es. `s : Student` è l'oggetto `s` di classe `Student`); per indicare una classe in generale, senza istanza, si scrive `: NomeClasse`.
- **Invocazione di un metodo:** freccia orizzontale continua dal chiamante al chiamato, con nome del metodo e parametri (es. `book(s, data)`).
- **Valore di ritorno:** freccia tratteggiata orientata verso sinistra, dal chiamato al chiamante, indicante l'eventuale valore restituito.
- **Barre di attivazione:** rettangoli verticali sulla **lifeline** (linea di vita) dell'oggetto, che indicano quale componente detiene l'esecuzione/computazione (in contesti `monothread/monotask`).



Elementi avanzati

Autochiamata (self-call)

- **Autochiamata:** una classe/oggetto invoca un proprio metodo interno (es. un controllo preliminare prima di un'operazione); si rappresenta con una freccia che parte dall'oggetto e si ripiega su se stessa.

Operatori e frame di interazione

Info

Frame

Rettangoli che racchiudono le chiamate per rappresentare logiche condizionali o iterative.

- **alt** (alternativa, if-else): frame diviso orizzontalmente da una linea tratteggiata; la condizione booleana è indicata tra parentesi quadre [condizione] — se vera si esegue la sequenza superiore, altrimenti quella inferiore.
- **opt** (opzione, if semplice): racchiude chiamate eseguite solo se una condizione è verificata, senza ramo di alternativa.
- **loop** (ciclo/iterazione): le chiamate nel blocco vengono ripetute per n volte o finché la condizione specificata resta vera.
- **ref** (reference): fa riferimento a un altro Sequence Diagram definito altrove, utile per mantenere i diagrammi leggibili, scomporre funzionalità complesse e riutilizzare sequenze comuni.

Chiamate sincrone e asincrone

- **Chiamata sincrona** (bloccante): freccia orizzontale con punta piena; il chiamante sospende la propria esecuzione finché il metodo chiamato non termina e restituisce il controllo.
- **Chiamata asincrona** (non bloccante): freccia orizzontale con punta aperta/mezza punta; il chiamante invoca il metodo e continua la propria computazione in parallelo con il chiamato.

Corrispondenza tra Sequence Diagram e codice

Dalla combinazione di Class Diagram e Sequence Diagram si ricava lo scheletro di codice funzionante delle classi coinvolte:

- Le **freccie entranti** in una classe ne definiscono i **metodi pubblici**/accessibili dall'esterno.

- Le **autochiamate** definiscono **metodi ausiliari/privati**.
- Le **invocazioni uscenti** indicano le operazioni eseguite **nel corpo del metodo**.

```
public class Exam {
    // autochiamata → metodo privato
    private boolean check(Student s, Date d) { ... }

    // freccia entrante → metodo pubblico
    public boolean saveBooking(Student s, Date d) {
        boolean ok = check(s, d);
        if (ok) {
            // invocazione uscente → chiamata nel corpo del metodo
            return dao.saveBooking(s, d);
        }
        return false;
    }
}
```

Associazione vs dipendenza

- **Associazione** (linea continua): l'altra classe è presente come **attributo di istanza permanente** nella classe (es. *Student* ha un attributo *Carriera*).
- **Dipendenza** (linea tratteggiata con freccia): la classe fa uso **temporaneo** di un'altra (es. ricevendola come **parametro di un metodo** o istanziandola come **variabile locale**), senza memorizzarla come attributo permanente.

Strumenti CASE

Info

Strumenti CASE

Per l'ingegneria del software conviene usare strumenti CASE dedicati (es. **Visual Paradigm**) anziché programmi di disegno generici (es. *Draw.io*, *PowerPoint*).

- **Coerenza semantica:** un vero strumento CASE gestisce il modello sottostante del software; aggiungere un metodo o un'interazione in un Sequence Diagram aggiorna e sincronizza automaticamente la struttura nel Class Diagram, garantendo la coerenza tra vista statica e vista dinamica.

Sintassi completa del Sequence Diagram

Example

